IESJC Benicarló
IES Joan Coromines

Unidad didáctica

Ficheros



Persistencia de datos en ficheros

Los **ficheros** son el método de almacenamiento de información más elemental. En última instancia, todos los métodos de almacenamiento, por sofisticados que sean, almacenan los datos en ficheros.

Los ficheros como medio de almacenamiento masivo de datos fueron relegados progresivamente en favor de las bases de datos relacionales. No obstante, las bases de datos relacionales utilizan internamente sistemas de gestión de ficheros para el almacenamiento de sus datos.

Tipos de ficheros según su contenido

Un fichero es simplemente una secuencia de *bytes*, por lo que puede almacenar cualquier tipo de información. Se identifica por su **nombre** y su **ubicación** dentro de una jerarquía de directorios. Cabe distinguir dos tipos:

- **Ficheros de texto**: contienen única y exclusivamente una secuencia de caracteres. Estos pueden ser caracteres visibles como letras, números, signos de puntuación, etc., y también espacios y separadores tales como tabuladores. Su contenido se puede visualizar y modificar con cualquier editor de texto.
- **Ficheros binarios**: son el resto de los ficheros. Pueden contener cualquier tipo de información. Hacen falta programas especiales para mostrar la información que contienen. Los programas también se almacenan en ficheros binarios.

Codificaciones para texto

Un **texto** es una secuencia de caracteres que se almacena en memoria o en cualquier dispositivo de almacenamiento como una secuencia de *bytes*.

Una **codificación** es un método para representar cualquier texto como una secuencia de *bytes*. El mismo texto, según la codificación empleada, se puede representar como una secuencia de *bytes* distinta.

La variedad de codificaciones es un problema cada vez menos importante en la práctica, debido a la implantación general de Unicode y su codificación **UTF-8**. Para las nuevas aplicaciones, hay que utilizar siempre UTF-8 para almacenar texto.

La clase File en Java

La clase **File** permite obtener información relativa a directorios y ficheros dentro de un sistema de ficheros y realizar diversas operaciones con ellos tales como borrar, renombrar, etc.

Categoría	Modificador / tipo	Métodos
Constructor		File (String ruta)
Consulta de propiedades	boolean	canRead(), canWrite(), canExecute()
	boolean	exists()
	boolean	isDirectory(), isFile()
	long	length()
	File	getParent(), getParentFile()

La clase File en Java

Categoría	Modificador / tipo	Métodos
Enumeración	String[]	list()
	File[]	listFiles()
Creación, borrado, renombrado	boolean	createNewFile()
	boolean	delete()
	boolean	renameTo()
	boolean	mkdir()

Ejercicio entregable #1

Escribe un programa en Java que muestre un listado de los ficheros y directorios que contiene el directorio desde el que se ejecuta el programa.

Si el directorio contiene otros directorios o ficheros, se deberán mostrar también.

Gestión de excepciones

Cualquier programa escrito en Java debe realizar una adecuada **gestión de excepciones**. Una excepción es un evento que ocurre durante la ejecución de un programa y que interrumpe su curso normal de ejecución. Una excepción podría producirse, por ejemplo, al dividir por cero.

```
package prueba;

public class ExcepcionesDivCero {
   public static int divide(int a, int b) {
      return a / b;
   }

   public static void main(String[] args) {
      int a, b;
      a = 5; b = 2;
      System.out.println(a + "/" + b + "=" + divide(a, b));
      b = 0;
      System.out.println(a + "/" + b + "=" + divide(a, b)); // Excepción
   }
}
```

Captura y gestión de excepciones

Cuando tiene lugar una excepción no gestionada se muestra un mensaje de error y se **interrumpe la ejecución** del programa. Por este motivo, el programa anterior no muestra el resultado de la última división.

La salida del programa indica el tipo de excepción que se ha producido, *ArithmeticException*. Para capturar la excepción se utiliza el bloque **try {} catch {}** y se maneja. El programa anterior quedaría así:

```
public static void main(String[] args) {
    int a, b;
    a = 5; b = 2;
    try {
        System.out.println(a + "/" + b + "=" + divide(a, b));
    } catch (ArithmeticException e){
        System.out.println("Error al dividir: " + a + "/" + b);
    }

    b = 0;
    try {
        System.out.println(a + "/" + b + "=" + divide(a, b));
    } catch (ArithmeticException e){
        System.out.println("Error al dividir: " + a + "/" + b);
    }
}
```

Distintos tipos de excepciones

En un bloque try-catch se pueden gestionar por separado distintos tipos de excepciones. Se debe incluir un manejador para ***Exception*** al final para que ninguna excepción se quede sin gestionar.

```
public static void main(String[] args) {
    // Rellenar array con números variados
    int nums[][] = new int[2][3];
    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 3; j++) {
        nums[i][j] = i + j;
      }
    }
    // Realizar cálculo para cada posición del array. Se producen excepciones de diversos tipos.
    for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 3; j++) {
        try {
          System.out.print("Segunda cifra de 5*nums[" + i + "][" + j + "] = ");
          System.out.println(String.valueOf(5 * nums[i][j]).charAt(1));
        } catch (ArithmeticException e) {
          System.out.println("ERROR: aritmético 5*" + nums[i][j] + "/" + j);
        } catch (ArrayIndexOutOfBoundsException e) {
          System.out.println("ERROR: No existe nums[" + i + "][" + j + "]");
        } catch (Exception e) {
          System.out.println("ERROR: de otro tipo al calcular segunda cifra de 5*" + nums[i][j] + "/" + j);
          e.printStackTrace();
        }
      }
    }
}
```

Ejercicio entregable #2

Se desea implementar un programa en Java que lea un fichero de texto que contiene números enteros, uno por línea. El programa debe:

1. Abrir el fichero cuyo nombre se indicará al principio del código.
2. Leer línea por línea el contenido del fichero. Para cada línea:
   a. Intentar convertir el contenido a un número entero.
   b. Calcular el cuadrado de dicho número y mostrarlo por pantalla.
3. Gestionar las siguientes posibles excepciones:
   a. FileNotFoundException: si el fichero no existe.
   b. IOException: si ocurre un error durante la lectura.
   c. NumberFormatException: si alguna línea no contiene un número válido.
   d. Una excepción genérica (Exception) para capturar cualquier otro error inesperado.
4. Finalmente, el programa debe mostrar un mensaje de cierre indicando que la ejecución ha terminado correctamente, independientemente de si hubo errores o no.

Enlace de ayuda: https://www.w3schools.com/java/java_files.asp

Excepciones lanzadas por un método

Si el compilador determina que un método de una clase puede originar un tipo de excepción pero no lo gestiona mediante un bloque *catch*, la compilación terminará con error. Si no se pone dicho bloque, la otra opción es añadir el modificador ***throws*** seguido de la clase de excepción.

```
public class ExcepcionesConThrows {

   public File creaFicheroTempConCar(String prefNomFich, char car, int numRep) throws IOException {
      File f = File.createTempFile(prefNomFich, "");
      FileWriter fw = new FileWriter(f);
      for (int i = 0; i < numRep; i++)
         fw.write(car);
      fw.close();
      return f;
   }

   public static void main(String[] args) {
      try {
         File ft = new ExcepcionesConThrows().creaFicheroTempConCar("AAAA_", 'A', 20);
         System.out.println("Creado fichero: " + ft.getAbsolutePath());
         ft.delete();
      } catch (IOException e) {
         System.err.println(e.getMessage());
      }
   }
}
```

Excepciones, liberación de recursos

Es muy frecuente que un bloque de programa de Java esté estructurado de la siguiente forma: **inicialización**, **cuerpo** y **finalización**. Estos bloques pueden estar dentro de un método de clase o de un bucle. La primera y última parte deben ejecutarse siempre, independientemente de los errores que puedan suceder durante la ejecución del cuerpo.

Pero es muy frecuente que cuando sucede algún error en mitad del cuerpo se quiera terminar inmediatamente la ejecución, de manera que no se ejecuta la parte final para finalización y liberación de recursos.

Para ello, esta última parte se coloca dentro de un bloque **finally {}**, de manera que se ejecutará justo antes de abandonar.

Excepciones, liberación de recursos

Inicialización y asignación de recursos

```
try {
        Cuerpo
} catch (Excepcion_Tipo_1 e1) {
        Gestión de excepción de tipo 1
} catch (Excepcion_Tipo_2 e2) {
        Gestión de excepción de tipo 2
} catch (Exception e) {
        Gestión del resto de tipos de excepciones
} finally {
        Finalización y liberación de recursos
}
```

Ejercicio entregable #3

Se desea implementar un programa en Java que copie el contenido de un fichero de texto a otro fichero. El programa debe:

1. Pedir el nombre del fichero de entrada y el nombre del fichero de salida como parámetros al inicio del programa.
2. Abrir el fichero de entrada en modo lectura y el de salida en modo escritura.
3. Leer línea por línea el contenido del fichero de entrada y escribirlo en el de salida.
4. Gestionar las siguientes excepciones:
    a. FileNotFoundException: si el fichero de entrada no existe.
    b. IOException: si ocurre un error durante la lectura o escritura.
    c. Exception: para cualquier otro error inesperado.
5. Usar un bloque finally para asegurarse de que los ficheros se cierren siempre, aunque ocurra un error.

Enlace de ayuda: https://www.w3schools.com/java/java_files.asp

Operaciones sobre ficheros

Independientemente del tipo de fichero las operaciones básicas sobre ficheros son en lo esencial iguales. El mecanismo de acceso a un fichero está basado en un **puntero** y en una zona de memoria que suele llamarse ***buffer***.

El puntero siempre apunta a un lugar del fichero o a una posición especial de fin de fichero. Esta se puede entender que está situada a continuación del último *byte* del fichero.

Operaciones sobre ficheros

Todas las clases para operar con ficheros disponen de las siguientes operaciones básicas:

- **Apertura**: antes de hacer nada con el fichero se debe abrir. Esto se hace al crear una instancia de una clase que se utilizará para operar con él.
- **Lectura**: mediante el método read(), consiste en leer contenidos del fichero para volcarlos a memoria y trabajar con ellos. El puntero se sitúa justo después del último carácter leído.
- **Salto**: mediante el método skip(). Consiste en hacer avanzar el puntero en un número determinado de *bytes* o caracteres hacia delante.
- **Escritura**: mediante el método write() escribe contenidos de memoria en un lugar determinado del fichero. El puntero se sitúa justo después del último carácter escrito.
- **Cierre**: el método close() permite cerrar el fichero para terminar.

Ejercicio entregable #4

Se desea implementar un programa en Java que trabaje con el siguiente fichero de texto utilizando las operaciones básicas de entrada/salida: elquijote.txt

El programa debe realizar las siguientes acciones:

- Leer todo el contenido del fichero y mostrarlo por la pantalla.
- Solicitar al usuario una palabra mediante teclado y contar cuántas veces aparece esa palabra en el texto (ignorando mayúsculas y minúsculas).
- Crear un fichero llamado resultado.txt en el que se escriba la palabra buscada y el número de veces que aparece en el Quijote.
- Cerrar todos los ficheros utilizados asegurándose de liberar recursos.

Enlace de ayuda: https://www.w3schools.com/java/java_files.asp